Studio Nx

AI 구현 설명 10 — 공통 헤더·푸터·로그인 모달 통합

2026-10-02

AI 구현 설명 10 — 공통 헤더·푸터·로그인 모달 통합 (HTML 조각)

항목	내용
작성	Claude (AI)
작성일	2026-10-02
작업	여러 페이지에 반복되는 헤더·푸터·로그인 모달을 한 파일씩으로 통합. 이전 문서: 07_auth_pages.md §3.7
브랜치	`feature/phase1/p1-06-join-login-modal`
원칙	JS는 원래 템플릿만이지만, 이 불러오기 코드는 **사용자 요청으로 AI가 구현** (화면 조립용 기반 코드)

---

1. 무엇을 만들었나

파일	내용
`static/partials/header.html`	공통 헤더 (로고, 섹션 메뉴, Sign In·Join Us / My Page·My Projects, 모바일 메뉴)
`static/partials/footer.html`	공통 푸터
`static/partials/login-modal.html`	로그인 모달 (C-2 시안, 소셜 버튼 자리)
`static/js/core/include.js`	**구현 완료.** `data-include` 자리를 조각 파일 내용으로 바꿈
`home.html`, `join.html`	헤더·푸터·모달 자리를 `<div data-include="...">` 한 줄씩으로
`pages/home.js`, `pages/join.js`	맨 처음 `await includePartials()` 후 다른 init 실행
`pages/join.css`	공통 헤더를 가입 페이지 모양으로 (섹션 메뉴·Join Us 숨김, 헤더 자리 확보)

2. 쓰는 법

새 페이지(마이페이지 등)를 만들 때:

```
<body>
  <a class="skip-link" href="#main">Skip to content</a>
  <div data-include="static/partials/header.html"></div>
  <main id="main"> ... </main>
  <div data-include="static/partials/footer.html"></div>
  <div data-include="static/partials/login-modal.html"></div>   ← 로그인 모달이 필요할 때만
  <script type="module" src="static/js/pages/새페이지.js"></script>
</body>
```

```
// pages/새페이지.js
import { includePartials } from '../core/include.js';
await includePartials();   // 이 줄 다음부터 헤더·모달 요소를 찾을 수 있음
```

- 헤더·푸터·모달을 고칠 때는 **`static/partials/`의 파일 하나만** 고치면 모든 페이지에 반영됩니다.
- 페이지마다 다른 부분은 그 페이지 CSS로 숨기거나 바꿉니다 (예: `join.css`가 섹션 메뉴 숨김).

3. 왜 이렇게 했나

3.1 다른 방법과 비교 (사용자와 결정)

방법	결과
A. HTML 조각 + 작은 불러오기 JS	✅ 적용 — 구조를 거의 바꾸지 않고, 마크업을 HTML 파일 그대로 관리
B. 웹 컴포넌트	마크업이 JS 문자열 안으로 들어가 디자인 수정이 불편
C. 서버 템플릿(Jinja2)	정적 제공 구조를 바꿔야 함
D. 빌드 스크립트	고칠 때마다 빌드, 원본·결과물 이중 관리

3.2 불러오기 코드 (`include.js`)

```
[data-include] 자리를 모두 찾음
 → 동시에 fetch (Promise.all)
 → <template> 에 넣어 HTML로 해석
 → 자리(div)를 그 내용으로 교체 (div 는 남지 않음 → 이전과 같은 DOM)
```

- `<template>`을 쓰는 이유: 조각 안의 HTML을 안전하게 해석하고, 조각 안 `<script>`는 실행하지 않습니다.
- 파일을 못 받으면 오류를 던져 콘솔에 바로 보입니다 (조용히 헤더가 빠지는 일 방지).
- 페이지 스크립트는 `type="module"`이라 `await includePartials()`를 맨 위에서 바로 쓸 수 있습니다.

3.3 헤더 링크를 `home.html#service` 형식으로

조각은 어느 페이지에서나 쓰이므로 `#service` 대신 `home.html#service`로 바꿨습니다. `home.html` 안에서는 같은 문서라 새로고침 없이 스크롤만 되고, 다른 페이지에서는 메인 페이지 해당 섹션으로 이동합니다.

3.4 깜빡임 대비

페이지	처리
`home.html`	헤더가 첫 화면 위에 겹쳐 있어(absolute) 늦게 들어와도 아래 내용이 밀리지 않음
`join.html`	헤더가 일반 흐름이라, 받기 전 자리(`min-height: var(--header-h)`)를 미리 확보

측정 결과 조각 3개가 페이지 로드(DOMContentLoaded) 후 **1~7ms 안에** 끼워집니다.

3.5 알아 둘 점

- **JS가 꺼져 있으면** 헤더·푸터·모달이 나오지 않습니다. 로그인·가입 자체가 JS(API 호출)로 동작하므로 이 서비스에서는 받아들일 만하다고 판단했습니다.
- `file://`로 HTML을 직접 열면 `fetch`가 막힙니다. 항상 개발 서버(`http://localhost:8000`)로 엽니다.

4. 검증 결과 (2026-10-02)

확인	결과
화면 회귀 (이전과 픽셀 비교)	`home.html` 전체 페이지(데스크톱·모바일), 로그인 상태 헤더, 모바일 메뉴 열림, 로그인 모달 열림 — 모두 **차이 0픽셀**
가입 페이지 회귀	P1-06 원래 캡처와 비교, 데스크톱·모바일 **차이 0픽셀**

확인	결과
동작	4가지 경우(메인 헤더, 메인 모바일 메뉴, 가입 헤더, 가입 카드 아래) 모두 클릭하면 모달이 열리고 이메일 칸에 커서
남은 자리	`[data-include]` 0개, 헤더·모달 각 1개
콘솔 오류	없음
디자인 자동 검사	파일·`home.html`·`join.html` 0건

5. 자주 받을 질문 대비

- **Q. 왜 서버 템플릿(Jinja2)을 쓰지 않았나?** 지금은 FastAPI가 `frontend/` 폴더를 그대로 제공합니다. 템플릿으로 바꾸면 페이지마다 라우트가 생기고 구조가 커집니다. 나중에 React로 옮길 계획이 있어 지금 구조를 크게 바꾸지 않았습니다.
- **Q. React로 옮길 때는?** `partials/`의 각 파일이 그대로 `<Header />`, `<Footer />`, `<LoginModal />` 컴포넌트 후보가 됩니다.
- **Q. 조각 안에 또 조각을 넣을 수 있나?** 지금 코드는 한 단계만 처리합니다. 필요해지면 끼운 뒤 `includePartials(새 내용)`을 다시 부르면 됩니다.